Nama : Mohamad Zaky Bahtiar Arifianto
Matkul : Praktikum Pemrogramman Berorientasi Obyek
Jurusan : Teknik Informatika
NRP : 3125522015

Tugas Pertemuan 12

BAB I: TUJUAN PRAKTIKUM & DASAR TEORI

Praktikum Pemrograman Berorientasi Obyek Modul 12 menitikberatkan pada implementasi persistensi data objek antara Random Access Memory (RAM) dan media penyimpanan berkas teks (.txt) permanen menggunakan pustaka modern Java NIO (java.nio.file.Files). Studi kasus yang dikembangkan adalah Sistem Manajemen Program Makan Bergizi Gratis (MBG) berbasis desktop JavaFX.

1.1 Tujuan Praktikum

- **Penyimpanan Objek Dinamis ke File:** Menyimpan kumpulan objek Menu dari memori (Collection) ke file teks (.txt) berformat terstruktur.
- **Rekonstruksi Objek dari File:** Membaca file teks dan membentuk kembali kumpulan objek domain lengkap dengan validasi integritas.
- **Separation of Concerns:** Memisahkan logika pembacaan/penulisan file ke class khusus (MbgFileRepository) agar tidak menumpuk di Controller.
- **Robust Exception Handling:** Menangani berkas tidak ditemukan, izin ditolak, dan format cacat menggunakan IOException tanpa crash.
- **Konsistensi State (Fail-Safe):** Memvalidasi seluruh baris ke koleksi sementara terlebih dahulu; koleksi utama pantang dihapus sebelum seluruh baris valid 100%.

1.2 Dasar Teori Persistensi File & Java NIO

Objek dalam Java Collection berada di dalam RAM selama aplikasi aktif dan bersifat volatil (hilang saat program ditutup). Agar data tersedia secara permanen saat program dijalankan kembali, data harus disimpan ke media penyimpanan sekunder (SSD/HDD). Java 7 memperkenalkan java.nio.file.Files yang menyediakan metode berkas modern dan efisien:

- **Path:** Merepresentasikan jalur lokasi berkas sistem secara fleksibel dan platform-independent.
- **Files.write():** Menulis sekumpulan string atau baris koleksi ke berkas fisik secara terpusat.
- **Files.readAllLines():** Membaca seluruh baris berkas teks secara langsung ke dalam List<String>.
- **StandardCharsets.UTF_8:** Menjamin encoding UTF-8 standar internasional agar tidak terjadi kerusakan karakter antar-OS.
- **IOException:** Menyampaikan kegagalan operasi berkas seperti berkas hilang atau akses ditolak.

BAB II: ARSITEKTUR FILE REPOSITORY & STRUKTUR DATA MBG

Dalam proyek MBG Modul 12, arsitektur sistem memisahkan tanggung jawab antara Controller antarmuka pengguna, Pengelola Koleksi Memori (PengelolaMenu), dan Penanggung Jawab Berkas Eksternal (MbgFileRepository).

2.1 Desain Format Berkas menu_mbg.txt

Berkas teks dirancang dengan format terstruktur satu baris mewakili satu objek paket menu, dengan pemisah delimiter pipa (|):

```
kode_menu|nama_menu|kalori_per_porsi|protein_gram|karbohidrat_gram|biaya_per_porsi
Contoh data aktual:
MNU-01|Paket A: Nasi Ayam Teriyaki & Tumis Sayur|680.0|28.5|82.0|15000.0
MNU-02|Paket B: Nasi Telur Balado & Sayur Sop|640.0|24.0|79.0|14000.0
```

2.2 Prinsip Separation of Concerns (Class Khusus MbgFileRepository)

Pemisahan tanggung jawab diwujudkan dengan menciptakan kelas MbgFileRepository. Controller UI JavaFX dan CLI Main tidak membaca atau menulis berkas secara manual. Controller hanya memanggil metode simpan(Path) atau muat(Path) pada MbgFileRepository. Jika format penyimpanan di masa mendatang ditingkatkan ke JSON atau database cloud, antarmuka pengguna tetap utuh.

2.3 Mekanisme Fail-Safe State Invariance

Saat memuat berkas, sistem menggunakan koleksi sementara (PengelolaMenu sementara = new PengelolaMenu()). Setiap baris dibaca, dipisah menggunakan .split("\\|", -1), divalidasi keunikan kode dan tipe datanya, lalu dimasukkan ke 'sementara'. Koleksi utama pantang dikosongkan (clear()) di awal. Koleksi utama HANYA digantikan setelah 100% baris terbukti valid tanpa error. Jika ada data tidak valid atau duplikat, exception dilemparkan dan koleksi lama di memori tetap aman.

BAB III: IMPLEMENTASI KODE PROGRAM UTAMA

3.1 Kode Class Khusus File I/O: MbgFileRepository.java

```
package P12_3125522015_MohamadZaky.utils;

import P12_3125522015_MohamadZaky.model.Menu;
import P12_3125522015_MohamadZaky.model.PengelolaMenu;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

public class MbgFileRepository {
    private final PengelolaMenu pengelolaMenu;

    public MbgFileRepository() { this(new PengelolaMenu()); }
    public MbgFileRepository(PengelolaMenu pengelolaMenu) {
        if (pengelolaMenu == null) throw new IllegalArgumentException("PengelolaMenu null");
        this.pengelolaMenu = pengelolaMenu;
    }

    public void simpan(Path lokasi) throws IOException {
        List<String> baris = new ArrayList<>();
        for (Menu menu : pengelolaMenu.getDaftarMenu()) {
            String kode = menu.getKodeMenu();
```

```
            String nama = menu.getNamaMenu();
            String kalori = String.valueOf(menu.getKaloriPerPorsi());
            String protein = String.valueOf(menu.getProteinGram());
            String karbo = String.valueOf(menu.getKarbohidratGram());
            String biaya = String.valueOf(menu.getBiayaPerPorsi());

            for (String nilai : new String[]{kode, nama, kalori, protein, karbo, biaya}) {
                if (nilai != null && (nilai.contains("|") || nilai.contains("\n") || nilai.contains("\r"))) {
                    throw new IllegalArgumentException("Kode/judul mengandung pemisah terlarang.");
                }
            }
            baris.add(kode + "|" + nama + "|" + kalori + "|" + protein + "|" + karbo + "|" + biaya);
        }
        Files.write(lokasi, baris, StandardCharsets.UTF_8);
    }

    public void muat(Path lokasi) throws IOException {
        List<String> baris = Files.readAllLines(lokasi, StandardCharsets.UTF_8);
        PengelolaMenu sementara = new PengelolaMenu();

        for (int i = 0; i < baris.size(); i++) {
            String[] bagian = baris.get(i).split("\\|", -1);
            if (bagian.length != 6) {
                throw new IOException("Format salah pada baris " + (i + 1));
            }
            try {
                double kalori = Double.parseDouble(bagian[2].trim());
                double protein = Double.parseDouble(bagian[3].trim());
                double karbo = Double.parseDouble(bagian[4].trim());
                double biaya = Double.parseDouble(bagian[5].trim());
                Menu menu = new Menu(bagian[0], bagian[1], kalori, protein, karbo, biaya);
                sementara.tambahMenu(menu);
            } catch (Exception e) {
                throw new IOException("Data tidak valid pada baris " + (i + 1) + ": " + e.getMessage(), e);
            }
        }
        // Ganti collection hanya setelah semua baris valid tanpa error
        pengelolaMenu.gantiSemua(sementara.getDaftarMenu());
    }
}
```

3.2 Pembaruan PengelolaMenu.java: Metode gantiSemua() dan Delegasi I/O

```
    public void gantiSemua(List<Menu> listBaru) {
        if (listBaru == null) throw new IllegalArgumentException("Daftar menu null");
        daftarMenu.clear();
        daftarMenu.addAll(listBaru);
    }

    public void simpan(Path lokasi) throws IOException {
        new MbgFileRepository(this).simpan(lokasi);
    }

    public void muat(Path lokasi) throws IOException {
        new MbgFileRepository(this).muat(lokasi);
    }
```

BAB IV: TABEL SKENARIO PENGUJIAN & LOG EKSEKUSI PROGRAM

Pengujian fungsional dilakukan mencakup seluruh skenario praktikum P12: simpan valid, muat ulang, file tidak ada, format baris rusak, ID duplikat, dan pembuktian keterjagaan state (State Invariance):

No	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
1	Simpan dua data valid ke file teks	File menu_mbg.txt berhasil dibuat berisi dua baris terstruktur delimiter '|'	File terbentuk sempurna (2 baris data menu)
2	Tutup program, lalu jalankan mode muat	Kedua objek Menu terbentuk kembali dari file ke dalam memori	Kedua menu berhasil direkonstruksi lengkap, size = 2
3	Muat file yang tidak ada (file_tidak_ada.txt)	Pesan kegagalan operasi file muncul elegan, program tidak crash	Tertangkap NoSuchFileException: 'file_tidak_ada.txt'
4	Muat file dengan format kolom sengaja dirusak	Pemuatan ditolak melempar IOException 'Format salah pada baris 1'	Ditolak: IOException 'Format salah pada baris 1'
5	Muat file berisi data dengan ID duplikat	Pemuatan ditolak melempar IOException bahwa kode menu sudah terdaftar	Ditolak: 'Data tidak valid baris 2: MNU-01 sudah terdaftar'
6	Uji konsistensi state (muat file rusak saat collection ada isinya)	Exception ditangkap; data koleksi utama lama tidak boleh hilang/terhapus	State Invariance terjamin: 1 menu lama tetap aman di memori
7	Muat file kosong (menu_kosong.txt)	File kosong dianggap valid; collection menjadi kosong secara sah (size = 0)	Berhasil memuat file kosong; total menu menjadi 0
8	Simpan data dengan delimiter terlarang ('|')	Ditolak melempar IllegalArgumentException sebelum penulisan berkas	Ditolak: 'Kode/judul mengandung pemisah terlarang'



Gambar 4.1: Log Eksekusi Verifikasi Otomatis Modul 12 pada Terminal PowerShell

BAB V: JAWABAN EVALUASI RESMI MODUL 12

Pertanyaan 1: Apa perbedaan data di collection (memori) dan data di file?

Jawaban: Data di dalam Collection (List<Menu>) berada pada memori utama (RAM) yang bersifat volatil (hilang seketika saat aplikasi ditutup atau komputer padam) dan direpresentasikan sebagai objek hidup yang

memiliki perilaku (method), tipe data kuat, dan keterkaitan relasi di memori. Sedangkan data di dalam file tersimpan pada media penyimpanan sekunder non-volatil (SSD/HDD) secara permanen dalam bentuk teks mentah (raw string bytes), sehingga data tetap bertahan melintasi siklus hidup program namun membutuhkan proses serialisasi dan deserialisasi untuk dapat diolah kembali.

Pertanyaan 2: Mengapa data dari file tetap harus divalidasi saat dimuat?

Jawaban: File eksternal merupakan sumber masukan yang berada di luar kendali langsung sistem (untrusted source). Berkas teks dapat diedit secara manual oleh pengguna lain melalui teks editor, rusak selama proses transfer, atau sengaja disisipi nilai tidak valid (seperti ID menu duplikat, format kalori bukan angka, atau nilai gizi negatif). Validasi ketat saat proses muat menerapkan prinsip Fail-Fast untuk menjamin agar objek korup tidak pernah menyusup ke dalam memori, sehingga kestabilan sistem dan kebenaran kalkulasi gizi program MBG tetap terlindungi.

Pertanyaan 3: Apa risiko mengosongkan collection utama sebelum seluruh baris file selesai diperiksa?

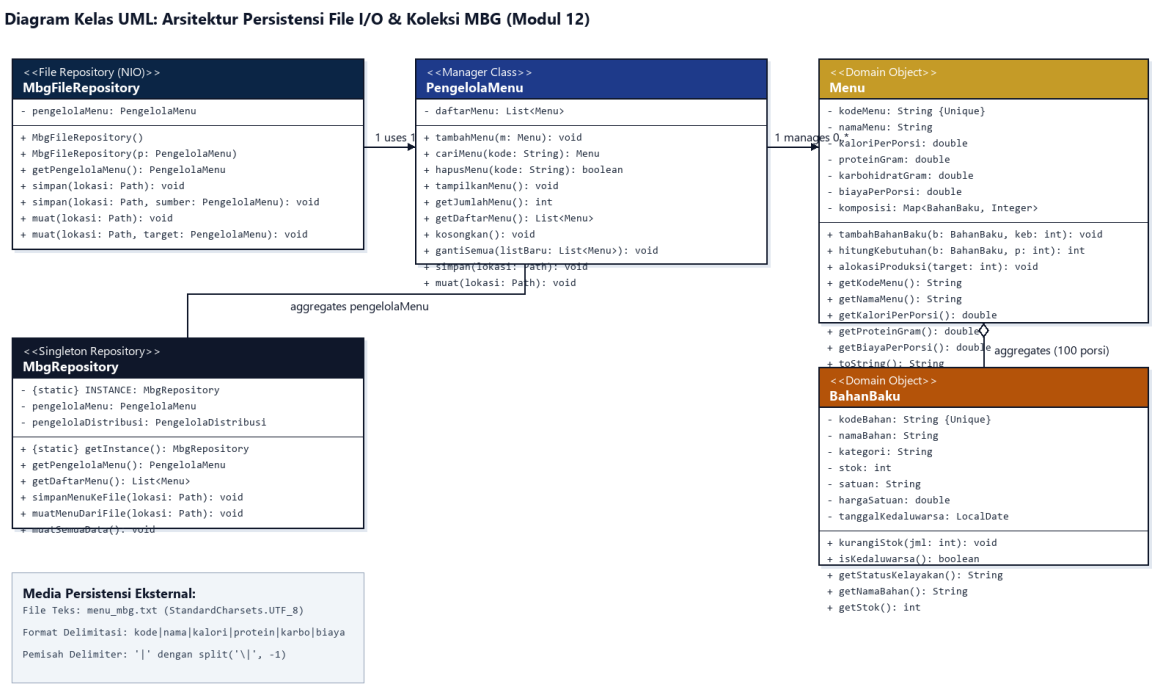
Jawaban: Risiko utamanya adalah terjadinya kehilangan data permanen (data loss on failure). Jika collection utama dikosongkan (clear()) di awal pemuatan, lalu terjadi kesalahan format atau duplikasi pada baris berikutnya di dalam berkas, proses muat akan terhenti seketika oleh exception. Akibatnya, data lama yang sebelumnya telah ada di memori lenyap dan data pengganti dari file gagal terbentuk. Dengan menampung objek ke koleksi sementara terlebih dahulu, prinsip atomisitas (State Invariance) terjaga: jika pembacaan gagal, koleksi lama tetap utuh tanpa modifikasi.

Pertanyaan 4: Mengapa fitur simpan harus dipanggil kembali setelah data di dalam aplikasi ditambah atau diubah?

Jawaban: Operasi penambahan, penyuntingan, atau penghapusan data di layar aplikasi hanya memodifikasi objek di dalam memori kerja (RAM). Collection Java tidak memiliki mekanisme sinkronisasi otomatis ke berkas fisik di disk. Oleh sebab itu, metode simpan() wajib dipanggil kembali untuk menimpa (overwrite) berkas fisik dengan kondisi mutakhir dari memori, sehingga perubahan terbaru tidak musnah saat aplikasi dimatikan.

BAB VI: PEMBARUAN UML CLASS DIAGRAM

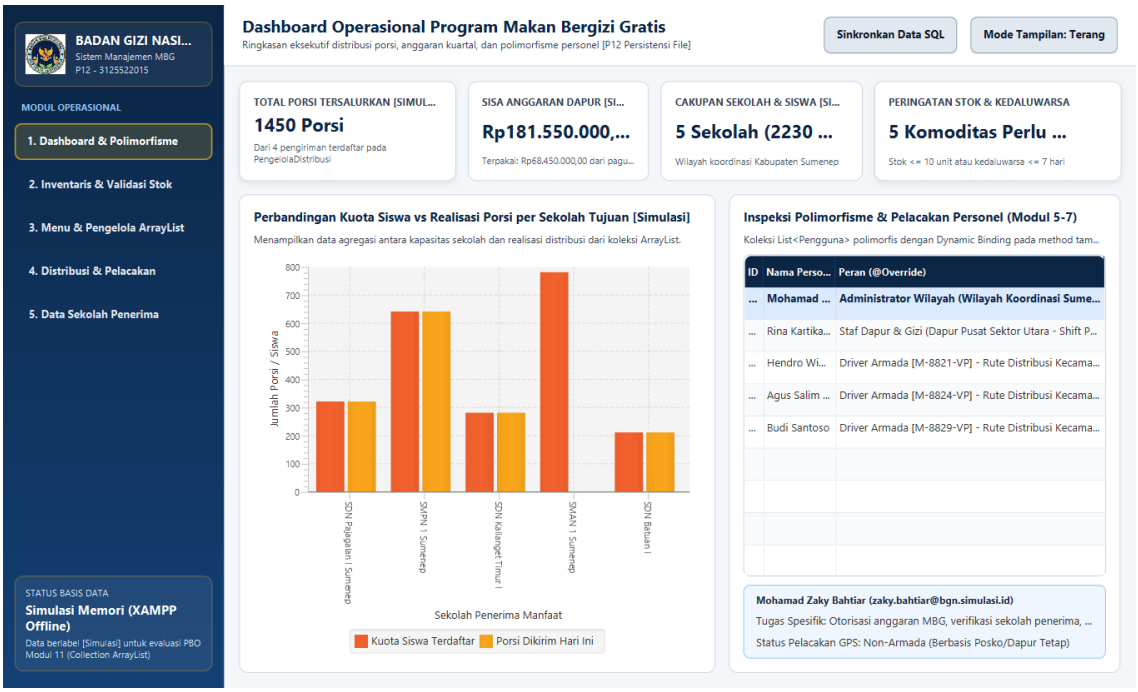
Diagram kelas berikut menggambarkan integrasi MbgFileRepository sebagai kelas khusus pengelola file I/O yang terhubung dengan PengelolaMenu dan model Menu dalam program MBG:



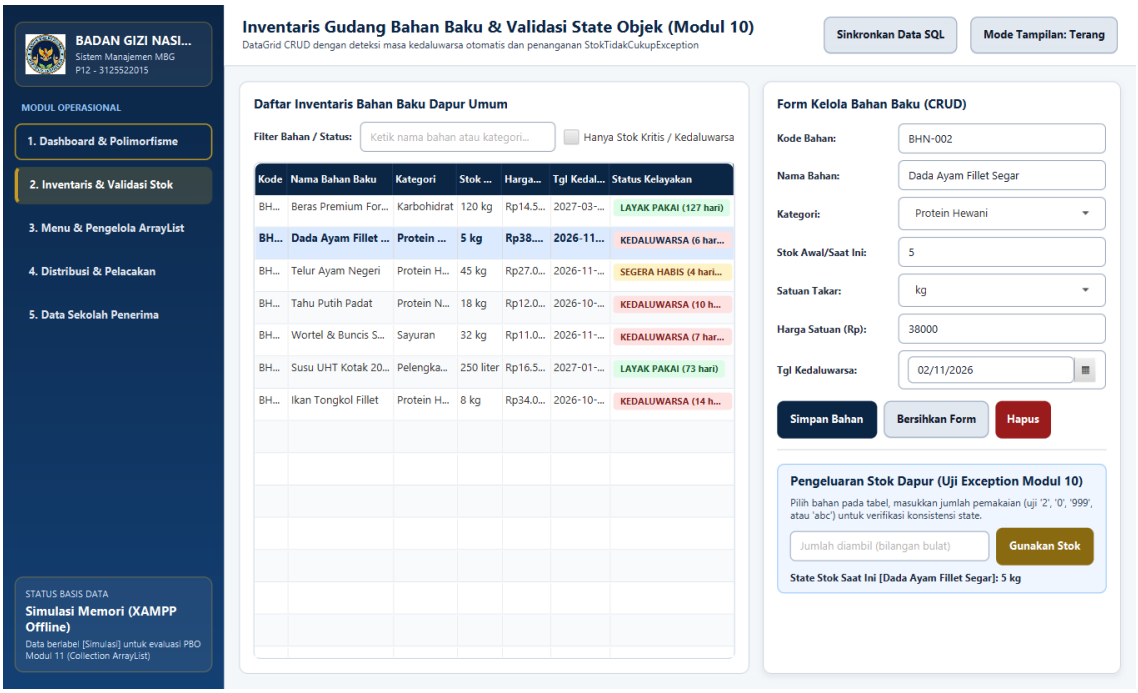
Gambar 6.1: Diagram Kelas UML Arsitektur Persistensi File I/O dan Koleksi MBG

BAB VII: DOKUMENTASI TAMPILAN ANTARMUKA APLIKASI DESKTOP JAVAFX

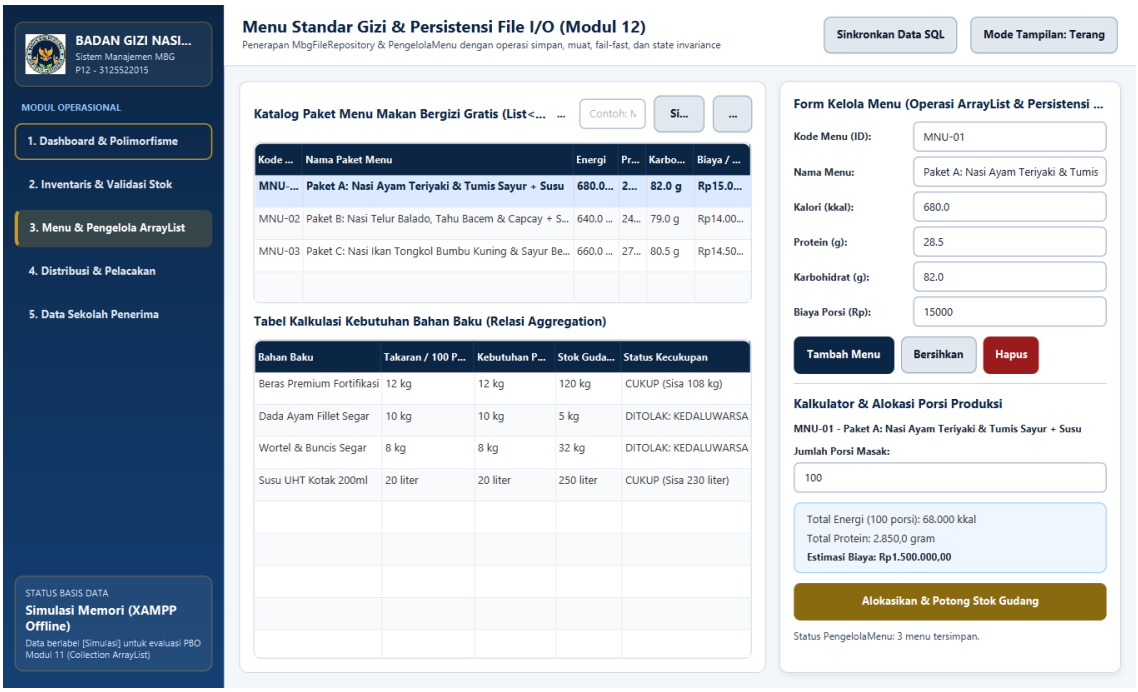
Berikut adalah dokumentasi tangkapan layar antarmuka pengguna Sistem Manajemen MBG P12 berbasis JavaFX yang telah mengintegrasikan modul kontrol File I/O:



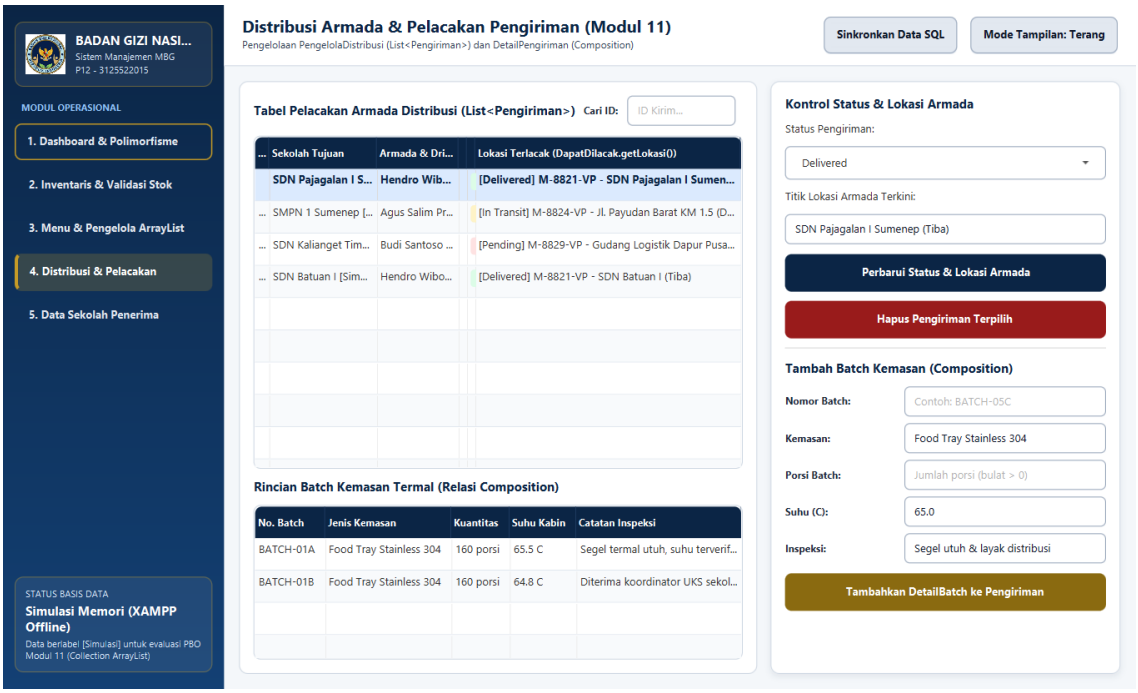
Gambar 7.1: Dashboard Operasional MBG, Ringkasan Porsi Terdistribusi & Personel



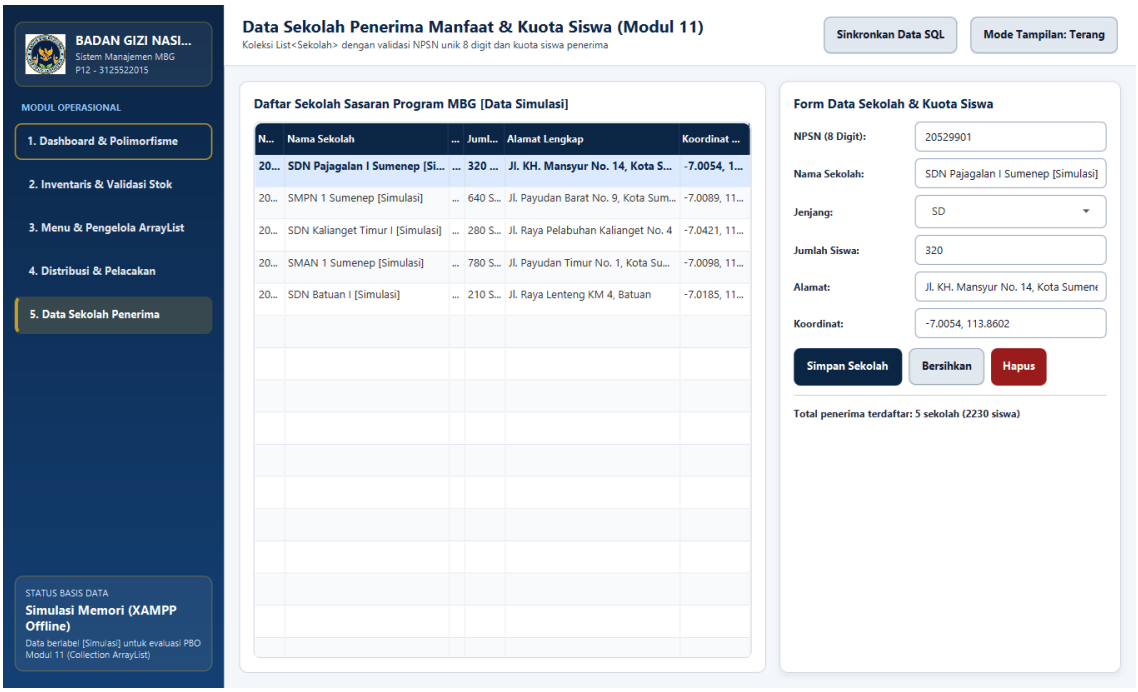
Gambar 7.2: Modul Inventaris Gudang Bahan Baku & Validasi Stok Makanan



Gambar 7.3: Modul Katalog Menu Gizi, Form CRUD, serta Tombol Simpan & Muat File (.txt)



Gambar 7.4: Modul Pelacakan Armada Distribusi & Komposisi Batch Kemasan



Gambar 7.5: Modul Data Sekolah Sasaran Penerima Manfaat MBG & Kuota Siswa

BAB VIII: KESIMPULAN

- **Efektivitas Persistensi File I/O:** Penggunaan java.nio.file.Files dan StandardCharsets.UTF_8 terbukti andal dalam menjembatani kebutuhan persistensi data antara memori volatil dengan berkas teks terstruktur.
- **Separation of Concerns:** Pemisahan logika I/O ke kelas MbgFileRepository menegakkan arsitektur bersih, independen, dan mempermudah perawatan sistem tanpa membebani Controller UI.
- **Ketahanan State Invariance:** Penggunaan koleksi penampung sementara (temporary collection) berhasil menjamin keutuhan data memori utama dari ancaman berkas rusak atau ID duplikat.
- **Integritas Format Delimitasi:** Validasi karakter delimiter '|' sebelum proses penulisan terbukti efektif mencegah tabrakan pemecahan token baris saat pembacaan ulang.
- **Kesiapan Operasional:** Sistem Manajemen MBG kini memiliki kemampuan menyimpan dan merekonstruksi data secara permanen lintas eksekusi aplikasi secara mulus.